

Homework 03

Justin Adrowski

Problem 1)

A)

```
1  # Most appropriate instruction format: I-type Instruction
2  # This is appropriate since cnt t0, t1 only includes 2
3  # register locations, which is characteristic of the I-type
4  # instruction
5
6
7  loop:
8  lb x7, 0(t0)
9  blt x7, x0, negative
10 addi t0, t0, 1
11 addi x6, x6, 1
12 j loop
13 negative:
14
15 # It's not included since it would be an I-type instruction, but it also writes
16 # two registers. No RISC-V instructions can write to two instructions within
17 # one instruction
18
19 ## this also wasn't included since the PC constantly gets re-evaluated within the loop,
20 ## so the entire loop couldn't be programmed within one instruction from the PC
```

B) Field-by-field

1)

Or t0, s0, a0					
R-type					
Funct7	Rs2	Rs1	Funct3	rd	opcode
0	a0 = 10	s0 = 8	6	t0 = 5	51
0000000	01010	01000	110	00101	0110011
0x00A462B3					

2)

addi t1, zero, 127				
I-type				
Imm	Rs1	Funct3	rd	opcode
127	0	0	t1 = 6	19
000001111111	00000	000	00110	0010011
0x07F00313				

3)

lw t2, 64(t0)				
I-type				
Imm	Rs1	Funct3	rd	opcode
64	t0 = 5	2	t2 = 7	3
000001000000	00101	010	00111	0000011
0x0402A383				

4)

sltu t3, t2, t1					
R-type					
Funct7	Rs2	Rs1	Funct3	rd	opcode
0	t1 = 6	t2 = 7	3	t3 = 28	51
0000000	00110	00111	011	11100	0110011
0x0063BE33					

5)

beq t3, zero, loop					
SB-type					
Imm(12,10:5)	Rs2	Rs1	Funct3	Imm(4:1,11)	opcode
-8	Zero = 0	t3 = 28	0	-8	99
1,111111	00000	11100	000	1100,1	1100011
0xFE0E0CE3					

6)

J begin		
IJ-type		
Imm(20, 10:1, 11, 19:12)	rd	opcode
-20	0	111
11111110110111111111	00000	1101111
0xFEDFF06F		

C) lui sequence

```

2  # two-instruction sequence
3  lui t0, 0xFEED2
4  addi t0, t0, 0x050
5
6  # 4-5 instruction sequence
7  addi t0, x0, -17    # 0xFFFFFEEF
8  slli t0, t0, 12     # 0xFFFEF000
9  addi t0, t0, -736   # 0xFFFEF000 + 0xFFFFFD20 = 0xFFFEED20
10 slli t0, t0, 8
11 addi t0, t0, 80

```

two-instruction sequence

1)

lui t0, 0xFEED20		
U-type		
Imm(31:12)	rd	opcode
FEED20	t0=5	111
111111101110110100100000	00101	0110111
0xFEED202B7		

2)

addi t0, t0, 0x050				
I-type				
Imm	Rs1	Funct3	rd	opcode
0x050	5	0	t0 = 5	19
000001010000	00101	000	00101	0010011
0x05028293				

five-instruction sequence

1)

addi t0, zero, -17				
I-type				
Imm	Rs1	Funct3	rd	opcode
-17	0	0	t0 = 5	19
111111101111	00000	000	00101	0010011
0xFE F00293				

2)

slli t0, t0, 12				
I-type				
Imm	Rs1	Funct3	rd	opcode
12	5	1	t0 = 5	19
000000001100	00000	001	00101	0010011
0x00C01293				

3)

addi t0, t0, -736				
I-type				
Imm	Rs1	Funct3	rd	opcode
-736	5	0	t0 = 5	19
110100100000	00101	000	00101	0010011
0xD2028293				

4)

slli t0, t0, 8				
I-type				
Imm	Rs1	Funct3	rd	opcode
8	5	1	t0 = 5	19
000000001000	00000	001	00101	0010011
0x00801293				

5)

addi t0, t0, 80				
I-type				
Imm	Rs1	Funct3	rd	opcode
80	5	0	t0 = 5	19
000001010000	00101	000	00101	0010011
0x05028293				

Problem 2)

a) Iterative Implementation

```

1  ### C-code
2
3  # include<stdio.h>
4  # include<stdlib.h>
5
6  # int main (void) {
7      # int N, Fib, i;    // amount of nums in sequence, fibonacci number
8      # int* arr; // array of nums
9      # printf("How many nums in seq:");
10     # scanf("%d", &N); // user input
11     # arr = malloc(N*sizeof(int));
12     # for (i = 0; i<N; i++) {
13         ## if(i>=2){
14             #   arr[i] = arr[i-1] + arr[i-2];
15             #   } else if (i==1){
16                 #   arr[i] = 1;
17             #   } else if (i==0){
18                 #   arr[0] = 0;
19             #   }
20             #   Fib = arr[i];
21             #   }
22     # free(arr)
23     # return 0;
24 # }
25
26 # RISC-V implementation

```

```

28 .data
29 prompt: .asciz "How many nums in sequence?\n"
30 result: .asciz "Fib = "
31 .text
32
33 main:
34     # printf(...prompt...);
35     la a0, prompt    # a0 = address of the prompt string
36     li a7, 4
37     ecall
38
39     # take user input (scanf("%d", &N);)
40     li a7, 5
41     ecall
42     mv s0, a0        # N is saved in s0
43
44     blez s0, exit    # if N == 0 -> nothing to compute, so exit
45
46     # allocate memory on stack (PUSH!) for arr according to input N
47     slli t0, s0, 2    # t0 = N*4 bytes
48     # Align stack sp to multiple of 16 bytes to match RISC-V memory allocation convention
49     addi t0, t0, 15   # adds number of bytes to >=16
50     andi t0, t0, -16  # ands with FFFFFFF0 to eliminate any non 16-multiple amount of bytes
51
52     mv s1, t0        # remember size to deallocate (POP!)
53     sub sp, sp, t0    # allocate
54     mv s2, sp        # s2 = base addr of arr
55
56     li t6, 1
57     sw x0, 0(s2)      # set arr[0] directly to zero to skip if statements within for loop
58     beq s0, t6, EqualsN    # N==1 -> arr[1] doesn't exist
59     sq t6, 4(s2)      # set arr[1] directly to one to skip if statements within for loop
60
61     li t2, 2          # have loop start at i = 2 if user input is at least 3 numbers
62     loop:
63         bge t2, s0, EqualsN
64         slli t3, t2, 2 # t3 = i*4
65         add t3, s2, t3 # t3 = &arr[i]
66         lw t4, -4(t3)  # t4 = arr[i-1]
67         lw t5, -8(t3)  # t4 = arr[i-2]
68         add t4, t4, t5 # t4 = arr[i-1]+arr[i-2]
69         sw t4, 0(t3)   # arr[i] = t4
70         addi t2, t2, 1 # i++
71         j loop         # jump back to start of loop
72     EqualsN:
73         # Fib = arr[N-1]
74         addi t3, s0, -1 # t3 = N-1
75         slli t3, t3, 2 # t3 = (N-1)*4
76         add t3, t2, t3 # t3 = &arr[N-1]
77         lw s3, 0(t3)
78
79         # print("Fib = %d\n", Fib)
80         la a0, result
81         li a7, 4      # print out text
82         ecall
83         mv a0, s3
84         li a7, 1      # print out integer
85         ecall
86         li a0, 10     # put '\n' in a0
87         li a7, 11     # print character
88         ecall
89
90         # Pop memory
91         add sp, sp, 1
92
93     exit:
94         li a7, 10     # exit
95         ecall
96

```

b) Recursive Implementation

```

1  # C code
2
3  # include<stdio.h>
4  # int fib(int n) {
5      ##    if (n==0) return 0;
6      ##    if (n==1) return 1;
7      #    return fib(n-1) + fib(n-2);}
8
9  # int main (void) {
10     #    int N;
11     #    printf("How many nums in sequence?\n");
12     #    scanf("%d", &N);
13     ##    if (N > 0)
14     #        printf("Fib = %d\n", fib(N-1));
15     #    return 0;}
16

```

```

17 # RISC-V Implementation
18 .data
19 prompt: .asciz "How many nums in sequence?\n"
20 result: .asciz "Fib = "
21 .text
22 j main      # start at main
23 fib:
24     addi sp, sp, -16
25     sw ra, 12(sp)
26
27     # base case 1: N == 2 -> Fib = 1
28     li t0, 2
29     beq a0, t0, base1
30     # base case 2: N == 1 -> Fib = 0
31     li t1, 1
32     beq a0, t1, base2
33
34     # recursive case: fib(n-1) + fib(n-2)
35     addi a0, a0, -1      # a0 = n-1 (argument for firstcall)
36     addi t2, a0, -1      # t2 = n-1 (arg for 2nd call)
37     sw t2, 4(sp)         # store n-2 to stack for 2nd call
38     jal ra, fib          # jump back to top (a0 = fib(n-1))
39     sw a0, 8(sp)         # save fib(n-1)
40     lw a0, 4(sp)         # load a0 = n-2
41     jal ra, fib          # new call where a0 = fib(n-2)
42     lw t1, 8(sp)         # t1 = fib(n-1)
43     add a0, t1, a0        # a0 = fib(n-1) + fib(n-2)
44     j done
45 base1:
46     li a0, 1
47     j done
48 base2:
49     li a0, 0
50
51 done:
52     lw ra, 12(sp)
53     addi sp, sp, 16
54     ret
55 main:
56     # printf(...prompt...);
57     la a0, prompt      # a0 = address of the prompt string
58     li a7, 4
59     ecall
60
61     # take user input (scanf("%d", &N);)
62     li a7, 5
63     ecall
64     mv s0, a0          # N is saved in s0
65     blez a0, exit
66
67     # Fib = fib(N)
68     mv a0, s0          # a0 (argument) is N
69     jal ra, fib        # call fib with a0 = N
70     mv s1, a0          # s1 = Fib
71
72     # print("Fib = %d\n", Fib);
73     la a0, result
74     li a7, 4          # print out text
75     ecall
76     mv a0, s3
77     li a7, 1          # print out integer
78     ecall
79     li a0, 10         # put '\n' in a0
80     li a7, 11         # print character
81     ecall
82
83 exit:
84     li a7, 10
85     ecall

```

C) Number of instructions

1) Iterative

of instructions

$N == 0 \Rightarrow 9$ instructions

$N == 1 \Rightarrow 32$ instructions (takes first beq branch)

$N == 2 \Rightarrow 18 + 16 = 34$ instructions (takes first pass of bge branch)

$N \geq 3 \Rightarrow 18 + 16 + (N-1) \cdot 9$ instructions (passes over bge $N-1$ times and executes the next 8 instructions $N-1$ times, so add 9 onto the $18+16$ $(N-1)$ times).

2) Recursive

of instructions

$N == 0 \Rightarrow 8$ instructions

$N = 1 \Rightarrow 32$ instructions

$N = 2 \Rightarrow 31$ instructions

$N \geq 3 \Rightarrow 28 \cdot F(N) + 29 \cdot F(N-1) + 5$